UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE FOR ROBOTICS II

PDDL & PDDL+

Assignment D2-V3:

Domestic Service Robot Mobile Manipulator

Author:

MAHDI BAGHBAN GHALEHCHI (S6741099)

June 11, 2026

1. Introduction

This report implements Assignment D2-V3: Domestic Service Robot - Mobile Manipulator. The assignment describes a humanoid or mobile manipulator preparing coffee in a kitchen where objects are distributed across locations such as a fridge, sink, and table. It explicitly requires the robot to both navigate and manipulate objects; navigation and manipulation must be modelled as separate actions, object access must depend on the robot location, and movement must not be merged with manipulation. For Q1, the assignment requires a basic PDDL model, one simple layout, one complex layout requiring multiple moves, and valid plans. For Q2, it requires a PDDL+ extension with a process modelling time consumption during movement, an event for deadline violation, and an explanation of how time affects planning decisions.

2. Q1-Basic PDDL Model

2.1 PDDL Domain Design

The Q1 domain is a classical PDDL model for the domestic service robot scenario. It represents a single mobile manipulator that prepares coffee in a discrete kitchen environment. The model is intentionally symbolic and planner-friendly: the kitchen is represented as a graph of named locations, objects are represented as symbolic items, and robot manipulation is represented through gripper-state predicates. The domain uses `:strips` and `:typing`, which means that states are described by Boolean facts and each action changes the state through preconditions and effects.

The domain defines `location` and `item` types, with `cup` and `ingredient` as subtypes of `item`. The `location` type is used for all kitchen places, such as `counter`, `sink`, `machine`, and `table`. The `item` type represents objects that can be manipulated by the robot, while `cup` and `ingredient` specialize this category for the coffee-preparation task.

The main purpose of the Q1 domain is to model navigation and manipulation separately. Navigation is represented only by the `move` action, which changes the robot location but does not change the state of any object. Manipulation is represented by actions such as `pick-up`, `put-down`, `fill-water`, `add-coffee-powder`, `brew-coffee`, and `serve-coffee`. These actions require the robot to already be at the correct location, so object access depends on navigation facts. This satisfies the requirement that movement and manipulation must not be merged into a single action.

The domain does not manually specify a fixed action sequence. Instead, the correct order is produced by the planner from the preconditions and effects of the actions. For example, the robot cannot brew the coffee unless the cup already has water and coffee powder. Therefore, the planner must first generate the actions that add these ingredients before applying `brew-coffee`.

The main components of the Q1 domain are summarized in the following tables: types, predicates, and action groups.

Type	Meaning
location	A discrete place in the kitchen graph, such as counter, sink, machine, or table.
item	A general manipulable object.
cup	A subtype of item representing the coffee cup.
ingredient	A subtype of item representing ingredients such as coffee powder.

Predicate group	Predicates	Purpose
Navigation	robot-at, connected	Encode the kitchen graph and the current robot location.
Manipulation	handempty, holding, at	Encode object location and gripper capacity.
Kitchen functionality	water-source, coffee-machine, serving-place	Make actions depend on the functional role of the current location.
Coffee state	empty, has-water, has-coffee-powder, brewed, served	Represent the causal preparation chain from empty cup to served coffee.

Action group	Actions	Purpose
Navigation	move	Moves the robot between connected locations. It changes only the robot location.
Object handling	pick-up, put-down	Allows the robot to pick and place objects only when it is at the same location as the object.
Coffee preparation	fill-water, add-coffee-powder, brew-coffee, serve-coffee	Represents the symbolic steps required to prepare and serve coffee.

2.2 Simple PDDL problem layout

The simple PDDL problem uses a compact kitchen layout in which the `counter` acts as the central hub. The robot, the cup, and the coffee powder are initially placed at the `counter`, while the final objective is to serve the prepared coffee at the `table`. Although the required objects are initially close to the robot, the task still requires navigation because different stages of coffee preparation must be performed at specific functional locations.

Because the preparation actions have location-based preconditions, any valid plan generated by the planner must include navigation between the relevant functional locations. For example, `fill-water` is only applicable at a water-source location, `brew-coffee` is only applicable at a coffee-machine location, and `serve-coffee` is only applicable at a serving-place location. Therefore, even in the simple layout, the resulting plan demonstrates the separation between navigation and manipulation: movement actions only update the robot location, while manipulation actions become applicable only after the robot has reached the required location.

The planner must therefore find a sequence of navigation and manipulation actions that transforms the initial state into the required goal state. The resulting valid plan is shown in the table below.

Initial state: robot, cup, coffee powder at counter

Goal: served cup1 at the table

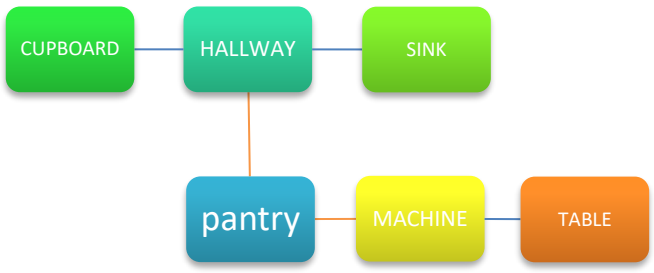
Valid plan for simple problem layout

#	Action
1	(pick-up cup1 counter)
2	(move counter sink)
3	(fill-water cup1 sink)
4	(move sink counter)
5	(put-down cup1 counter)
6	(pick-up coffee-powder counter)
7	(add-coffee-powder coffee-powder cup1 counter)
8	(put-down coffee-powder counter)
9	(pick-up cup1 counter)
10	(move counter machine)
11	(brew-coffee cup1 machine)
12	(move machine counter)
13	(move counter table)
14	(serve-coffee cup1 table)

2.3 Complex PDDL problem layout

The complex PDDL problem uses a more distributed kitchen layout than the simple instance. The robot initially starts in the `hallway`, while the final objective is to serve the prepared coffee at the `table`. The required objects are also distributed across different locations: the cup is located in the `cupboard`, and the coffee powder is located in the `pantry`. Therefore, the planner must solve both the coffee-preparation task and the navigation task created by the kitchen topology.

Because object access depends on the robot location and the robot has only one gripper, the planner must generate a longer sequence that coordinates navigation, object handling, and coffee-preparation actions. This makes the plan longer than in the simple layout, since the robot must respect the available connections between locations while satisfying the logical dependencies of the coffee-preparation process. The resulting valid plan is shown in the table below.



Initial state: robot at hallway, cup at cupboard, coffee powder at pantry

Goal: served cup1 at the table

Valid plan for complex layout

#	Action
1	(move hallway cupboard)
2	(pick-up cup1 cupboard)
3	(move cupboard hallway)
4	(move hallway sink)
5	(fill-water cup1 sink)
6	(move sink hallway)
7	(move hallway pantry)
8	(put-down cup1 pantry)
9	(pick-up coffee-powder pantry)
10	(add-coffee-powder coffee-powder cup1 pantry)
11	(put-down coffee-powder pantry)
12	(pick-up cup1 pantry)
13	(move pantry machine)
14	(brew-coffee cup1 machine)
15	(move machine table)
16	(serve-coffee cup1 table)

3. Q2 - PDDL+ Model

3.1 Why PDDL+ is needed

The Q2 domain extends the classical PDDL model by introducing time and autonomous world evolution. In Q1, movement is instantaneous: the `move` action directly removes the robot from one location and places it at another. However, the assignment requires time consumption during movement and an event for deadline violation. Therefore, the PDDL+ model replaces the instantaneous `move` action with a time-based movement mechanism.

In PDDL+, the model uses three different kinds of behavior. Actions are controlled by the planner, processes represent continuous changes controlled by the world, and events are automatic instantaneous changes that fire when their preconditions become true. This distinction is the core of the Q2 model. The planner does not directly choose when a movement finishes; it only starts the movement using the `start-move` action. The world then advances the movement progress over time through the `movement-progress` process and automatically completes the movement with the `finish-move` event when enough progress has been made.

Manipulation actions remain symbolic and instantaneous in this model. This is a deliberate abstraction because the assignment specifically asks for a process modelling time consumption during movement. Therefore, timing is introduced for navigation, while actions such as `pick-up`, `put-down`, `fill-water`, `add-coffee-powder`, `brew-coffee`, and `serve-coffee` remain discrete symbolic operations.

The following table summarizes the main PDDL+ constructs used in the Q2 domain, including movement actions, manipulation actions, numeric functions, processes, and events.

Construct group	Elements	Purpose
Navigation action	`start-move`	Starts time-consuming navigation between connected locations. It activates `moving`, stores the origin and destination, resets `move-progress`, and sets the required movement duration.
Manipulation actions	`pick-up`, `put-down`, `fill-water`, `add-coffee-powder`, `brew-coffee`, `serve-coffee`	Represent the symbolic coffee-preparation steps. These actions remain instantaneous in Q2 because the assignment specifically focuses on time consumption during movement.
Time functions	`elapsed-time`, `deadline`	Store the total mission time and the latest acceptable completion time.
Movement functions	`move-progress`, `current-move-duration`, `move-duration ?from ?to`	Store the progress of the current movement, the required duration of the active movement, and the travel time for each graph edge.
Processes	`mission-clock`, `movement-progress`	`mission-clock` continuously increases `elapsed-time` while the mission is running. `movement-progress` continuously increases `move-progress` while the robot is moving.
Events	`finish-move`, `deadline-violation`	`finish-move` automatically completes movement when progress reaches the required duration. `deadline-violation` automatically marks failure when the deadline is exceeded.

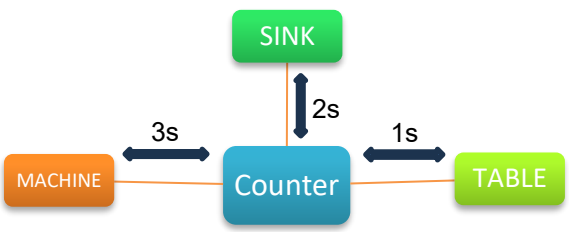
3.2 Simple PDDL+ problem layout

The simple PDDL+ problem uses the same basic kitchen structure as the simple classical PDDL problem. The robot, the cup, and the coffee powder initially start at the `counter`, while the final objective is to serve the prepared coffee at the `table`. The symbolic coffee-preparation requirements are the same as in Q1: water must be added at a water-source location, brewing must be performed at the coffee-machine location, and serving must be completed at the serving-place location.

The main difference from the classical PDDL version is that movement is no longer instantaneous. Instead of using the classical `move` action, the PDDL+ model uses `start-move` to begin navigation. While the robot is moving, the `movement-progress` process continuously increases `move-progress` over time. When the required movement duration is reached, the `finish-move` event automatically places the robot at the destination.

In this simple PDDL+ instance, travel durations are defined for each connected edge: moving between `counter` and `sink` takes 2 time units, moving between `counter` and `machine` takes 3 time units, and moving between `counter` and `table` takes 1 time unit. The deadline is set to 12 time units. The generated valid timed plan serves the coffee at time 12, so the `deadline-violation` event does not fire.

This problem shows how the same symbolic task from Q1 changes when time is introduced. The logical order of coffee preparation remains the same, but the validity of the plan now also depends on whether the required movements can be completed before the deadline. The resulting valid timed plan is shown in the table below.



Initial state: robot, cup, coffee powder at counter

Goal: served cup1 at the table

Valid plan for PDDL+ simple layout:

Time	Planner output
0.0	(pick-up cup1 counter)
0.0	-----waiting----- [1.0]
1.0	(start-move counter sink)
1.0	-----waiting----- [3.0]
3.0	[event] (finish-move counter sink)
3.0	(fill-water cup1 sink)
3.0	(start-move sink counter)
3.0	-----waiting----- [5.0]
5.0	[event] (finish-move sink counter)
5.0	(put-down cup1 counter)
5.0	(pick-up coffee-powder counter)
5.0	(add-coffee-powder coffee-powder cup1 counter)
5.0	(put-down coffee-powder counter)
5.0	(pick-up cup1 counter)
5.0	(start-move counter machine)
5.0	-----waiting----- [8.0]
8.0	[event] (finish-move counter machine)
8.0	(brew-coffee cup1 machine)
8.0	(start-move machine counter)
8.0	-----waiting----- [11.0]
11.0	[event] (finish-move machine counter)
11.0	(start-move counter table)
11.0	-----waiting----- [12.0]
12.0	[event] (finish-move counter table)
12.0	(serve-coffee cup1 table)

Note: After `(pick-up cup1 counter)`, the planner output includes a one-unit idle waiting step before `(start-move counter sink)`. This waiting step is not an event and does not represent manipulation duration. Manipulation actions remain instantaneous; only movement is modelled as time-consuming. The plan is still valid because the coffee is served exactly at the deadline, before any `deadline-violation` event is triggered.

3.3 Complex PDDL+ problem layout

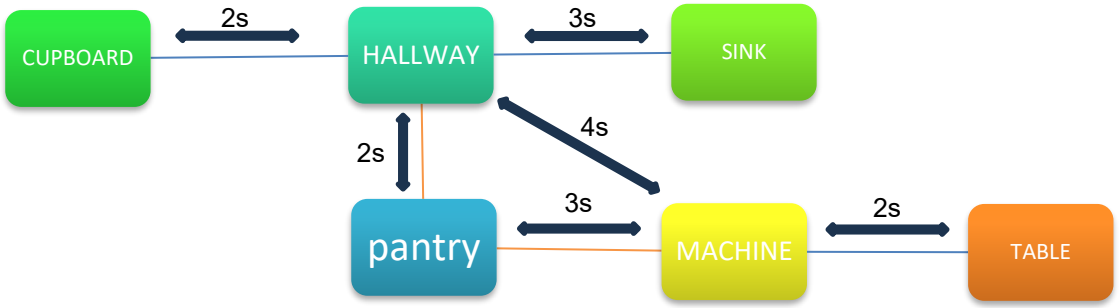
The complex PDDL+ problem uses the same distributed kitchen structure as the complex classical PDDL problem, but movement is now time-consuming. The robot initially starts in the `hallway`, while the final objective is to serve the prepared coffee at the `table`. The cup is located in the `cupboard`, and the coffee powder is located in the `pantry`. Therefore, the planner must solve the same navigation and manipulation problem as in the classical complex layout, but now under temporal constraints.

The main difference from the classical complex problem is that each movement has a duration. The robot does not instantly move from one location to another. Instead, each movement starts with the `start-move` action, progresses over time through the `movement-progress` process, and is completed automatically by the `finish-move` event when the required duration is reached.

The complex PDDL+ instance has a deadline of 18 time units. Therefore, a plan is valid only if it achieves the serving goal before the `deadline-violation` event is triggered. In the generated valid timed plan, the coffee is served at time 18, so the deadline is not exceeded, and the `deadline-violation` event does not fire.

This instance demonstrates how time affects planning decisions. In the classical model, a plan may be logically valid if it eventually reaches the goal. In the PDDL+ model, however, a logically correct plan can still become invalid if the accumulated movement time exceeds the deadline. Therefore, the

planner must consider not only reachability and action preconditions, but also whether the task can be completed on time. A valid timed plan that satisfies these constraints is shown in the table below.



Initial state: robot at hallway, cup at cupboard, coffee powder at pantry

Goal: served cup1 at the table

Valid plan for PDDL+ complex layout

Time	Planner output
0.0	-----waiting----- [1.0]
1.0	(start-move hallway cupboard)
1.0	-----waiting----- [3.0]
3.0	[event] (finish-move hallway cupboard)
3.0	(pick-up cup1 cupboard)
3.0	(start-move cupboard hallway)
3.0	-----waiting----- [5.0]
5.0	[event] (finish-move cupboard hallway)
5.0	(start-move hallway sink)
5.0	-----waiting----- [8.0]
8.0	[event] (finish-move hallway sink)
8.0	(fill-water cup1 sink)
8.0	(start-move sink hallway)
8.0	-----waiting----- [11.0]
11.0	[event] (finish-move sink hallway)
11.0	(start-move hallway pantry)
11.0	-----waiting----- [13.0]
13.0	[event] (finish-move hallway pantry)
13.0	(put-down cup1 pantry)
13.0	(pick-up coffee-powder pantry)
13.0	(add-coffee-powder coffee-powder cup1 pantry)
13.0	(put-down coffee-powder pantry)
13.0	(pick-up cup1 pantry)
13.0	(start-move pantry machine)
13.0	-----waiting----- [16.0]
16.0	[event] (finish-move pantry machine)
16.0	(brew-coffee cup1 machine)
16.0	(start-move machine table)
16.0	-----waiting----- [18.0]
18.0	[event] (finish-move machine table)
18.0	(serve-coffee cup1 table)

Note: The planner output includes idle waiting steps, including an initial one-unit wait before `(start-move hallway cupboard)`. These waiting steps are not events and do not represent manipulation duration. Manipulation actions remain instantaneous, and only movement is modelled as time-consuming. The plan is still valid because the coffee is served exactly at the deadline, before any `deadline-violation` event is triggered.

3.4 Invalid long-route demonstration

The same complex instance can become invalid when a longer navigation route is considered. In the long-route case, the robot follows the route from `pantry` to `machine` through `hallway` instead of using the direct connection. This increases the accumulated movement time, and the serving action would only be reached at time 21. Since the deadline is 18, the `deadline-violation` event is triggered before the coffee can be served. As a result, the goal cannot be achieved in this timed execution. This demonstrates that, in PDDL+, a plan can be logically correct but still invalid if it violates the temporal deadline.

Time	Planner output
0.0	-----waiting----- [1.0]
1.0	(start-move hallway cupboard)
1.0	-----waiting----- [3.0]
3.0	[event] (finish-move hallway cupboard)
3.0	(pick-up cup1 cupboard)
3.0	(start-move cupboard hallway)
3.0	-----waiting----- [5.0]
5.0	[event] (finish-move cupboard hallway)
5.0	(start-move hallway sink)
5.0	-----waiting----- [8.0]
8.0	[event] (finish-move hallway sink)
8.0	(fill-water cup1 sink)
8.0	(start-move sink hallway)
8.0	-----waiting----- [11.0]
11.0	[event] (finish-move sink hallway)
11.0	(start-move hallway pantry)
11.0	-----waiting----- [13.0]
13.0	[event] (finish-move hallway pantry)
13.0	(put-down cup1 pantry)
13.0	(pick-up coffee-powder pantry)
13.0	(add-coffee-powder coffee-powder cup1 pantry)
13.0	(put-down coffee-powder pantry)
13.0	(pick-up cup1 pantry)
13.0	(start-move pantry hallway)
13.0	-----waiting----- [15.0]
15.0	[event] (finish-move pantry hallway)
15.0	(start-move hallway machine)
15.0	-----waiting----- [18.0001]
18.0001	[event] (deadline-violation)
19.0	[event] (finish-move hallway machine) autonomous movement completion may still occur, but the mission has already failed
19.0	(brew-coffee cup1 machine) ; not executable after deadline-violation
19.0	(start-move machine table) ; not executable after deadline-violation
21.0	[event] (finish-move machine table) ; not executable after deadline-violation
21.0	(serve-coffee cup1 table) ; not executable because deadline-missed is true and mission-running is false

Note: After the `deadline-violation` event is triggered, the mission is no longer running, so the following agent actions are not executable in the actual model. The autonomous `finish-move` event may still complete a movement that had already started, but the task has already failed. The remaining rows are included only to illustrate that the longer route would reach the serving step too late.

4. Discussion

4.1 Integration of Navigation and Manipulation

The integration between navigation and manipulation is achieved by making manipulation actions depend on navigation-related predicates. For example, `pick-up` requires both `(robot-at ?l)` and `(at ?i ?l)`, which means that the robot can only pick up an object when it is located at the same place as that object. Therefore, the planner must first generate navigation actions that bring the robot to the required location before any manipulation action becomes applicable.

The same principle is used for the coffee-preparation actions. The `fill-water` action requires the robot to be at a water-source location while holding the cup, `brew-coffee` requires the robot to be at a coffee-machine location, and `serve-coffee` requires the robot to be at a serving-place location. This directly implements the assignment requirement that object access and manipulation must depend on the robot location.

4.2 Abstraction Gap Between Symbolic and Physical Motion

The model is deliberately symbolic. It does not compute robot trajectories, grasp poses, arm configurations, collision-free paths, or object geometry. Instead, physical feasibility is abstracted through symbolic predicates such as `robot-at`, `at`, `connected`, and `holding`.

This represents the task-motion gap discussed in the course materials. At the planning level, the robot reasons with discrete locations and logical facts, but real execution would require continuous motion planning, collision checking, grasp planning, and low-level control. Therefore, the PDDL and PDDL+ models capture the high-level task structure, while the detailed physical execution is left outside the scope of this assignment.

5. References:

- Fulvio Mastrogiovanni, *Planning Fundamentals*, `Part 2 - Planning fundamentals.pdf`.
- Fulvio Mastrogiovanni, *Advanced Planning Concepts*, `Part 3 - Advanced planning concepts.pdf`.
- Omar Kashmar / TA material, *PDDL Advanced Examples - Class 2*, `PDDL_Class2.pdf`.
- *Examples Explanation*, `Examples Explanation.pdf`.
- Omar Kashmar / TA material, *PDDL Introduction through 5 Classical Examples*, `PDDL_Introduction_1.pdf`.